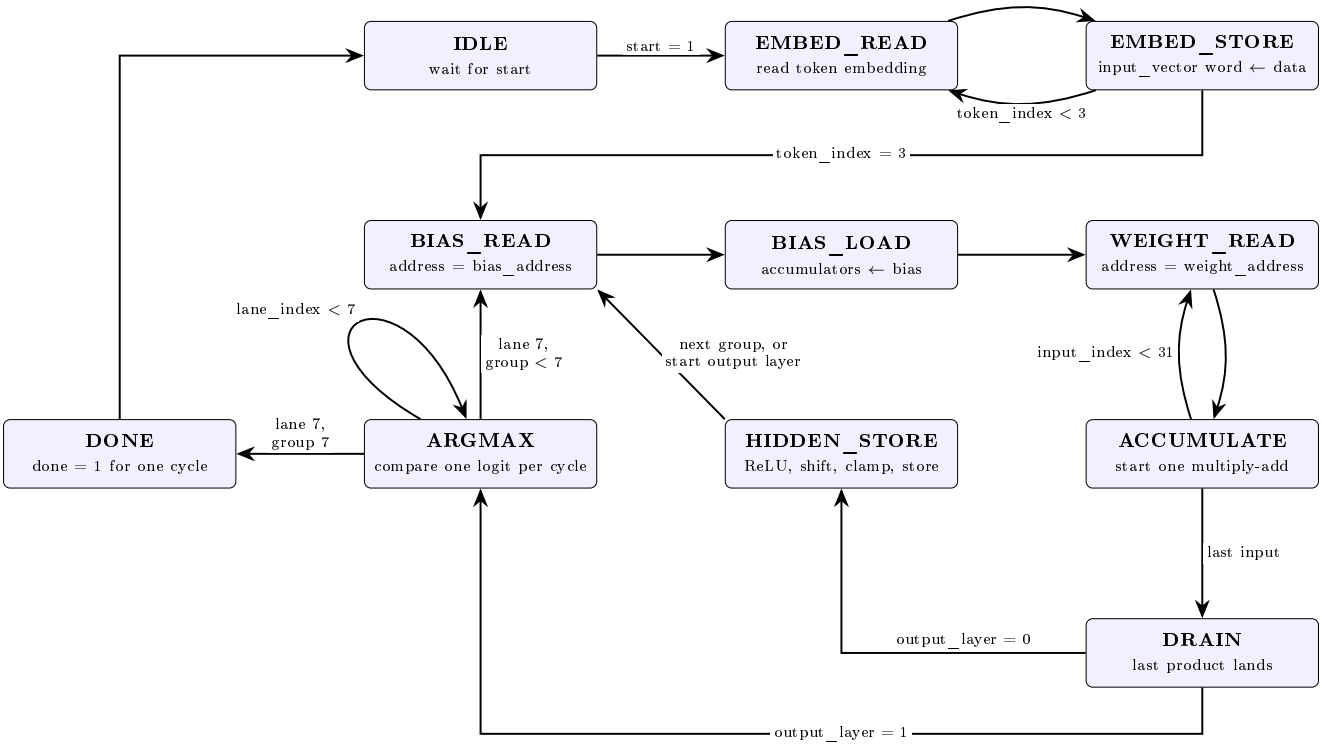
Tiny FPGA LM: inference FSM and parallel MAC lanes

Arty A7-100T, 100 MHz. One FSM sequences the embedding lookup, both fully connected layers, and the argmax. The same 8-lane MAC array and the same states serve both layers; only the `output_layer` register changes.

1. Inference FSM



State transition table

State	Condition	Next state	**Register updates at the clock edge**
IDLE	start = 0	IDLE	token_index, group_index, input_index, lane_index, weight_address, bias_address, output_layer ← 0 (every cycle spent in IDLE); argmax_clear = 1
IDLE	start = 1	EMBED_READ	same clearing; argmax_clear = 1 holds best_value at -2^{31} and best_index at 0
EMBED_READ	always	EMBED_STORE	embedding ROM latches the word at address token[token_index]
EMBED_STORE	token_index < 3	EMBED_READ	input_vector word token_index ← embedding ROM output (8 × INT8); token_index ← token_index + 1
EMBED_STORE	token_index = 3	BIAS_READ	input_vector word 3 ← embedding ROM output
BIAS_READ	always	BIAS_LOAD	bias ROM latches the word at bias_address
BIAS_LOAD	always	WEIGHT_READ	accumulator$_i$ ← bias$_i$ for all 8 lanes; bias_address ← bias_address + 1
WEIGHT_READ	always	ACCUMULATE	weight ROM latches the word at weight_address. The activation is selected from input_vector (output_layer = 0) or hidden_buf (output_layer = 1) using input_index, and mac_array registers it
ACCUMULATE	input_index < 31	WEIGHT_READ	each lane registers activation · weight$_i$; one clock later it adds that product into accumulator$_i$; weight_address ← weight_address + 1; input_index ← input_index + 1
ACCUMULATE	input_index = 31	DRAIN	same multiply-add; input_index ← 0
DRAIN	output_layer = 0	HIDDEN_STORE	no register updates: waits one clock so the last product reaches the accumulators
DRAIN	output_layer = 1	ARGMAX	same wait
HIDDEN_STORE	group_index < 3	BIAS_READ	hidden_buf byte group ← clamp(ReLU(accumulator$_i$) ≫ 4) for 8 lanes; group_index ← group_index + 1
HIDDEN_STORE	group_index = 3	BIAS_READ	same store; output_layer ← 1; group_index ← 0

State	Condition	Next state	**Register updates at the clock edge**
ARGMAX	lane_index < 7	ARGMAX	if accumulator[lane_index] > best_value: best_value ← it, best_index ← {group_index, lane_index}; lane_index ← lane_index + 1
ARGMAX	lane_index = 7, group_index < 7	BIAS_READ	same compare; lane_index wraps from 7 to 0; group_index ← group_index + 1
ARGMAX	lane_index = 7, group_index = 7	DONE	same compare
DONE	always	IDLE	done = 1 this cycle; predicted_token = best_index

Registers and memory layout

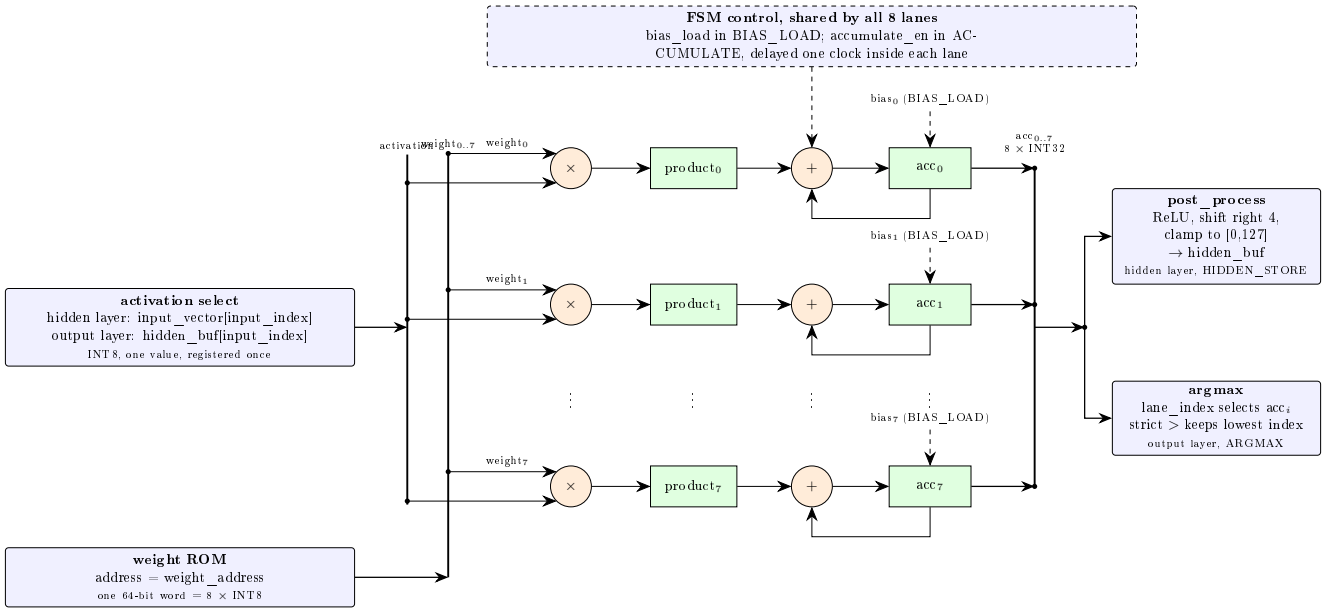
Register	**Bits**	**Meaning**
state	4	current FSM state
output_layer	1	0 = hidden layer (32→32), 1 = output layer (32→64)
token_index	2	which of the 4 context tokens is being embedded
group_index	3	which group of 8 output neurons (0–3 in the hidden layer, 0–7 in the output layer)
input_index	5	which of the 32 inputs within a group
lane_index	3	which lane's logit ARGMAX is examining
weight_address	9	next weight ROM word, 0 to 383; counts up and is not reset between layers
bias_address	4	next bias ROM word, 0 to 11
best_value	32 signed	largest logit so far
best_index	6	token ID of the largest logit so far (the output byte)

Weight ROM: 384 words of 64 bits (8 × INT8). Words 0–127 are the hidden layer (word = group·32 + input), words 128–383 are the output layer. Because the FSM visits words in order, the address is a plain counter with no multiplier. Word bits [8i+7:8i] hold $W[8 \cdot \text{group} + i][\text{input}]$ and go to lane i. The Python weight export must use this ordering.

Bias ROM: 12 words of 256 bits (8 × INT32), words 0–3 for the hidden layer and 4–11 for the output layer. Embedding ROM: 64 words of 64 bits (8 × INT8).

Latency: 8 cycles of embedding, 4 × 68 cycles for the hidden layer, 8 × 75 for the output layer, plus DONE, 881 cycles or 8.8 μs. One UART byte at 115200 baud takes 87 μs, so the sequencing cost is negligible.

2. Eight parallel MAC lanes



In an ACCUMULATE cycle, lane i multiplies the registered activation by its weight$_i$ and stores the 16-bit product in its product register; one clock later it performs $acc_i \leftarrow acc_i + product_i$. The activation (INT8) is registered once and shared by all eight lanes, and weight$_i$ (INT8) is that lane's own slice of the weight word. The weight ROM output is itself registered, so the activation and the weight word meet at the multipliers in the same ACCUMULATE cycle. The product is INT8 × INT8 → 16-bit signed, sign-extended into the 32-bit signed accumulator. The product register is what lets the multiplier map onto a DSP slice and the design meet 100 MHz; the cost is the DRAIN state, one extra clock after the last input of each group. In BIAS_LOAD the accumulator is loaded with the lane's 32-bit bias instead of the sum. The lanes never exchange data with each other, which is what distinguishes this from a systolic array.